

SecuRAG

Cybersecurity RAG Chatbot

A project understanding guide — architecture, design rationale, and interview prep for the RAG pipeline you built.

How to use this guide: Read Sections 1–3 to understand what the project is and why it's built this way. Use Section 4 as interview prep — each entry pairs a question you're likely to get with a strong, concise answer you can say out loud. Section 6 is your action plan for adding real weight to the project before you show it to anyone.

§	Section	What it covers
1	What This Project Actually Is	The one-sentence pitch and why RAG fits this domain
2	Architecture at a Glance	The 7-stage pipeline, visualized
3	Component Deep-Dive	Every stage — what it does, why it exists, what breaks without it
4	Interview Prep — Q&A	Common questions with ready-to-say answers
5	Evaluation & Metrics	How to prove it works, not just claim it
6	Action Plan	The concrete steps to add real weight, in priority order
7	Glossary	Every term, defined plainly

1 What This Project Actually Is

The one-sentence pitch:

A chatbot that answers cybersecurity questions by retrieving relevant passages from real policy documents and security frameworks, then generating an answer grounded in — and citing — those passages, instead of relying on what a language model memorized during training.

Why RAG, specifically, for this domain

Three properties of cybersecurity/compliance content make it a strong RAG use case rather than just an LLM chatbot:

- **Freshness matters.** CVEs, policies, and frameworks change. An LLM's training data goes stale; retrieval pulls from documents that can be updated independently.
- **Wrong answers are costly.** A hallucinated compliance requirement or made-up control ID isn't just embarrassing — it can create real liability. Grounding + citations turns unverifiable claims into checkable ones.
- **The content is dense and structured.** Control IDs (AC-2), category codes (A01:2021), and framework hierarchies reward exact-match retrieval alongside semantic search — which is why this project uses hybrid retrieval rather than semantic search alone.

What makes this more than a tutorial clone

Most RAG tutorials stop at "embed some text, do a vector search." This project's differentiators, which map directly onto interview talking points, are: heterogeneous ingestion (unstructured policy docs + structured JSON in one pipeline), structure-aware chunking (not fixed-size), hybrid retrieval (semantic + keyword, fused), cross-encoder re-ranking, citation-forced generation, contextual retrieval enrichment, and a separate retrieval-vs-generation evaluation harness.

2 Architecture at a Glance

1. RAW DOCUMENTS	Markdown policies · PDFs · Structured JSON (NIST, OWASP, MITRE-style)
2. INGESTION	document_loader.py — parses each format into one common shape: {text, source, section, doc_type}
3. CHUNKING	chunker.py — splits along document structure, not arbitrary size; attaches source/section header to every chunk
4. (OPTIONAL) CONTEXTUAL ENRICHMENT	contextual_enrichment.py — LLM call per chunk adds document-level context before embedding
5. EMBEDDING	embedder.py — BAAI/bge-small-en-v1.5, local & offline, converts text to vectors
6. VECTOR STORE	chroma_store.py — persisted Chroma collection, stores vectors + metadata
7. HYBRID RETRIEVAL	hybrid_search.py — semantic search + BM25 keyword search, merged via Reciprocal Rank Fusion
8. RE-RANKING	reranker.py — cross-encoder re-scores the candidate pool for precision
9. GENERATION	answer_generator.py — Claude API, citation-forced prompt, refuses to guess
10. CHAT UI	streamlit_app.py — shows the answer and its sources

Read this top to bottom and you can already answer "walk me through your architecture" — that single question opens almost every project-review conversation.

Two entry points that tie it together

`build_index()` runs once (or whenever source documents change): stages 2 → 6 above.

`SecuRAGPipeline.answer(query)` runs per question: stages 7 → 9. This split matters because it's the answer to "why is this fast at query time?" — the expensive work (parsing, embedding an entire corpus) happens once, not per question.

3 Component Deep-Dive

For each stage: what it does, the one design decision worth being able to defend, and what breaks if you skip it.

Ingestion

What it does: Parses PDFs/Markdown/JSON into one common record shape.

Key decision: Different loaders per format, unified output schema.

Why it matters: Downstream stages would need to know about every source format — this isolates that complexity to one file.

Chunking

What it does: Splits documents into retrievable pieces, along section boundaries.

Key decision: Structure-aware, not fixed character count; every chunk gets a [source | section] header.

Why it matters: Fixed-size chunking can split a heading from its body text — you'd retrieve a fact with no idea what it's about.

Embedding

What it does: Converts text into vectors that capture meaning, not just words.

Key decision: Local BGE model — offline, no per-call API cost.

Why it matters: Without embeddings, search is purely keyword-based and misses paraphrased questions.

Vector Store

What it does: Persists vectors + metadata; finds nearest neighbors fast.

Key decision: Chroma — embedded, no separate server process to run.

Why it matters: Without it you'd recompute embeddings and re-scan every chunk on every query — fine for 50 chunks, unusable at scale.

Hybrid Retrieval

What it does: Combines semantic search with BM25 keyword search via Reciprocal Rank Fusion.

Key decision: RRF instead of hand-tuned score weighting — simple, no manual calibration needed.

Why it matters: Semantic-only search can miss exact codes like "AC-2" or "CVE-2024-1234"; keyword-only misses paraphrased intent.

Re-ranking

What it does: A cross-encoder re-scores the top ~15-20 candidates for precision.

Key decision: Two-stage retrieve-then-rerank instead of one expensive pass over everything.

Why it matters: Running the accurate-but-slow cross-encoder over the whole corpus per query wouldn't scale; skipping it entirely leaves you with the initial, noisier ranking.

Contextual Enrichment (optional)

What it does: LLM writes a short context blurb per chunk before embedding.

Key decision: Applied only to unstructured docs, cached per source document to limit API calls.

Why it matters: Without it, a chunk that reads fine in a long document can lose meaning once pulled out of context.

Generation

What it does: Claude API turns retrieved chunks + question into a cited answer.

Key decision: Prompt forces a citation per claim and explicitly permits "I don't know."

Why it matters: Without citation-forcing, the model can blend retrieved facts with memorized (possibly wrong or outdated) knowledge, and you can't tell which is which.

Evaluation

What it does: Measures retrieval and generation quality separately, against labeled questions.

Key decision: Retrieval hit-rate and generation groundedness are tracked as two distinct numbers.

Why it matters: Without separating them, a wrong answer gives you no signal on whether to fix chunking/retrieval or fix the prompt.

4 Interview Prep — Q&A

Read through these once, out loud. You don't need to memorize the wording — you need the shape of each answer so you can say it in your own words under pressure.

Q: Walk me through the architecture.

A: Documents get ingested and normalized into one shape regardless of source format, then chunked along their natural structure rather than fixed size. Chunks are embedded and stored in a vector database. At query time, I run hybrid retrieval — semantic search plus BM25 keyword search, fused with Reciprocal Rank Fusion — then re-rank the top candidates with a cross-encoder for precision, and finally generate a citation-grounded answer with Claude.

Q: Why RAG instead of fine-tuning?

A: Fine-tuning bakes knowledge into model weights — expensive to update and doesn't give you traceable citations. RAG keeps the knowledge in documents I can add, edit, or remove independently, and every answer can point back to the exact source, which matters a lot in a compliance-adjacent domain where an unverifiable answer is a liability.

Q: Why hybrid retrieval instead of just vector search?

A: Security queries mix natural language with exact identifiers — a control ID like AC-2, a CVE number, an acronym like MFA. Semantic search is great at intent but can under-weight a rare exact token; BM25 keyword search catches those exact matches. I fuse both rankings with Reciprocal Rank Fusion so a document doing well in either method still surfaces, and one doing well in both gets boosted further.

Q: Why re-rank after retrieval instead of just taking the top-k from search?

A: Initial retrieval scores each candidate independently against the query in a shared vector space — fast, but approximate. A cross-encoder looks at the query and one candidate together in a single pass, which is far more accurate but too slow to run over an entire corpus. So I retrieve a wider candidate pool cheaply, then re-rank only those with the cross-encoder.

Q: How do you stop the model from hallucinating?

A: Two layers: retrieval only surfaces real chunks from real documents, and the generation prompt explicitly requires a citation for every claim and instructs the model to say it doesn't know rather than guess when the retrieved context doesn't cover the question. I evaluate this directly — generation groundedness is measured separately from retrieval quality specifically to catch cases where the right chunk was retrieved but the model ignored or misused it.

Q: How do you know it actually works? What are your metrics?

A: I built a labeled eval set of question/expected-source pairs and measure two things separately: retrieval hit rate — did the correct source show up in the top-k retrieved chunks — and generation groundedness — did the final answer actually cite that source. Separating them tells you whether a wrong answer is a retrieval problem or a generation problem, which point at completely different fixes.

Q: What was the hardest part / what would you do differently?

A: [Personalize this once you've actually run it locally — see Section 6. A strong version of this answer names a real bug you hit, e.g. a chunking edge case, a PDF that extracted badly, or a query where hybrid retrieval outperformed pure semantic search, with the before/after numbers.]

Q: How would this scale to a much larger document set?

A: Chroma runs embedded and is fine for a single-machine demo, but I'd swap it for Qdrant or Weaviate for a production deployment with millions of chunks — the retrieval interface is written so that's a swap in one file, not a rewrite. I'd also add metadata filtering so queries can be scoped to a specific framework or document type before the vector search even runs, cutting the search space.

Q: What's contextual retrieval, and why did you add it?

A: It's a technique where, before embedding, an LLM generates a short description of what each chunk is about in the context of its full document, and that description gets prepended before embedding. It fixes cases where structure-aware chunking alone still isn't enough — a chunk that's technically self-contained but has lost some implicit context from its surroundings. It's a one-time cost per chunk at index-build time, not per query, so it doesn't affect latency.

Q: Why did you separate retrieval evaluation from generation evaluation?

A: Because they fail for completely different reasons and need completely different fixes. If retrieval hit rate is low, the problem is in chunking, embedding, or search — no amount of prompt engineering fixes that. If retrieval succeeds but the generated answer doesn't cite the right source, the problem is in the generation prompt. Measuring them together would hide which one is actually broken.

5 Evaluation & Metrics

A project that only shows working examples reads as untested. A project with real, run metrics — even mediocre ones — reads as engineered. This section is what to actually run and report.

The two numbers that matter

Metric	What it tells you	If it's low, the fix is in...
Retrieval hit rate	Did the correct source chunk appear in the top-k retrieved results at all?	Chunking, embedding model, or search — not the prompt.
Generation groundedness	Given the right chunks were retrieved, did the model's answer actually cite them?	The generation prompt or the model's instruction-following.

How to run it

```
python -m src.pipeline --build
python -m eval.evaluate
```

This runs all 20 labeled questions in `eval/test_questions.json` against your built index and prints both hit rates, plus writes per-question detail to `eval/results.json`.

What to do with the numbers

- Put the real percentages in your README — replace any placeholder language.
- If hit rate is below ~80%, look at which questions failed and why — often a chunking or section-boundary issue you can point to specifically.
- Re-run after changing one variable at a time (chunk size, embedding model, candidate_pool) and keep a small table of results — this table is worth more in an interview than any amount of architecture description.

Common trap: reporting only "it works on my test questions" without showing the number, or without showing what happens on questions it gets wrong. Both hit rate *and* a couple of honest failure examples are what make an eval section credible.

6 Action Plan — Adding Real Weight

In priority order. Each tier assumes the previous one is done — don't skip to Tier 4 polish before Tier 1 is real.

Tier 1 — Make it real

- Install dependencies and build the index locally — this is where real bugs surface.
- Run `eval/evaluate.py` for real and put the actual numbers in your README.
- Feed in real source documents (full NIST PDF, OWASP cheat sheets, MITRE JSON export).

Tier 2 — Fix what breaks with messy data

- Improve PDF extraction if real documents extract badly (swap `pypdf` for `unstructured`).
- Tune chunk size empirically — run eval at 400 / 800 / 1200 chars, keep the comparison table.
- Compare embedding models (`bge-small` vs `bge-large`) on hit rate vs latency.

Tier 3 — Deploy it somewhere real

- Deploy the Streamlit app publicly (Streamlit Community Cloud is free).
- Confirm docker compose up works from a clean clone, on a different machine.

Tier 4 — Depth that shows judgment

- Write a short design-decisions section: why hybrid retrieval, why RRF, what changes at 10x scale.
- Add a failure-mode section — 2-3 real queries where the system is wrong, with your explanation.
- Benchmark hybrid+rerank against a naive cosine-similarity baseline on the same eval set.

If you only do one thing: run the eval script for real, and put the resulting numbers plus a 3-sentence "what I learned tuning chunk size" writeup in the README. That single artifact is what turns this from "a RAG scaffold" into "a RAG system I built and evaluated."

7 Glossary

Term	Definition
RAG (Retrieval-Augmented Generation)	Pairing a search step with a generation step: retrieve relevant text, then have an LLM answer using that text as grounding.
Chunk	A piece of a document small enough to retrieve individually and fit in a prompt, ideally still meaningful on its own.
Embedding	A vector (list of numbers) representing a piece of text's meaning; similar meanings produce similar vectors.
Vector database	A database optimized for storing embeddings and finding the nearest ones to a query vector, fast, at scale.
Cosine similarity	A measure of how similar two vectors' directions are — the standard way to compare embeddings.
BM25	A classic keyword-ranking algorithm; scores documents by term overlap with the query, weighted by term rarity.
Hybrid search	Combining semantic (vector) search and keyword (BM25) search so both intent-matching and exact-term-matching are covered.
Reciprocal Rank Fusion (RRF)	A simple way to merge two ranked lists into one, based on each item's rank position rather than raw incomparable scores.
Cross-encoder	A model that scores a (query, document) pair jointly, more accurate but slower than comparing pre-computed embeddings — used for re-ranking a small candidate set.
Re-ranking	A second, more precise scoring pass over an initial set of retrieved candidates, to improve final ordering.
Hallucination	An LLM generating fluent, confident text that is factually wrong — a core risk RAG is designed to reduce.
Groundedness	Whether a generated answer is actually supported by the retrieved source text, rather than invented or memorized.
Contextual retrieval	Prepending an LLM-generated summary of a chunk's document-level context before embedding it, so the chunk's meaning survives being taken out of the document.
Top-k	The number of top-ranked results kept at a given stage (e.g. "retrieve the top 15, then re-rank down to the top 5").
Hit rate (retrieval)	The fraction of test questions where the correct source chunk appeared in the retrieved results.

Guide generated alongside the SecuRAG project. Update Section 5's numbers and Section 4's "hardest part" answer once you've run this locally — that's what makes this document (and the project) genuinely yours.